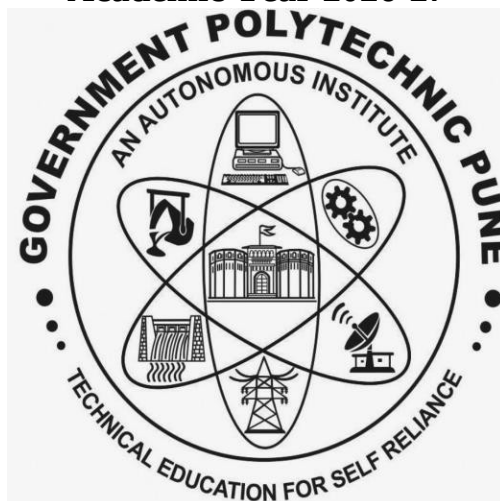


DEPARTMENT OF COMPUTER ENGINEERING
Academic Year 2026-27



A Micro Project Report On

“Medicare – Hospital Management and Doctor Appointment Application”

Submitted By

Name	Enrollment No
Pravin Kadam	2406088

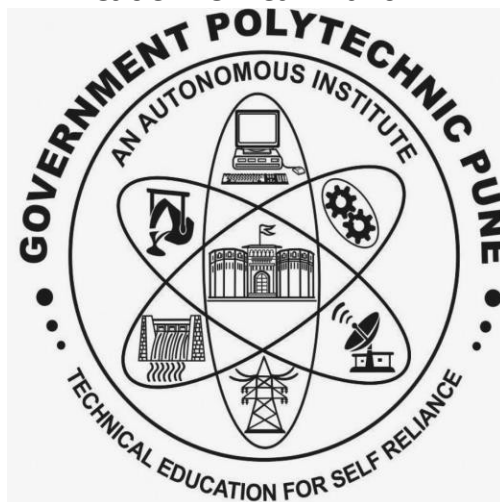
Guided by:

Smt.H.S.Pawar

Lecturer, Government Polytechnic Pune

DEPARTMENT OF COMPUTER ENGINEERING

Academic Year-2026-27



CERTIFICATE

This is to certify that the micro-project work entitled “**Medicare – Hospital Management and Doctor Appointment Application**” is a bonafide work carried out by

Name	Enrollment No
Pravin Kadam	2406088

Of class **Third Year** in partial fulfilment of requirement for completion of course **Mobile Application Development (CM51203)** of Diploma in Computer Engineering from **Government Polytechnic, Pune**. The report has been approved as it satisfies the academic requirements in respect of microproject work prescribed for the course.

Smt.H.S.Pawar

(Microproject Guide)

Smt.J.R.Hange

(Head of Department)

Dr.R.K.Patil

(Principal)

ACKNOWLEDEMENT

We extend our deepest gratitude to **Dr. Rajendra Patil**, our esteemed Principal, whose unwavering support and encouragement inspired us throughout the journey of our project, **“Medicare – Hospital Management and Doctor Appointment Application.”** His vision and guidance paved the path for its successful completion.

Special recognition is due to **Mrs. J. R. Hange**, our dedicated Head of Department, for her invaluable insights and constant motivation, which played a pivotal role in shaping this endeavour.

A profound thank-you to **Smt. H.S.PawarMa’am**, our respected mentor, whose boundless patience and consistent guidance were the cornerstones of our project journey. Her valuable advice and keen insights were instrumental in overcoming challenges and achieving our objectives.

Furthermore, we acknowledge that the completion of this project was a testament to our dedication and perseverance. While we received immense support and direction from our mentors and faculty, the execution and culmination of this work were solely our own.

In closing, we express our heartfelt gratitude to all those who contributed directly or indirectly to the success of this project. This work stands as a reflection of teamwork, determination, and the pursuit of academic excellence.

ABSTRACT

Medicare – Hospital Management and Doctor Appointment Application is an Android-based mobile application developed to provide a convenient and organized solution for hospital management and doctor appointment booking. Finding a suitable doctor and booking an appointment can be difficult, especially when patients have to visit the hospital and wait in long queues. Medicare aims to simplify this process through a mobile application.

The application provides functionality for patients to search doctors by name and specialty, view doctor details, book appointments and view their prescriptions. It also provides dedicated functionality for doctors to accept, reject and complete appointments and issue digital prescriptions, and for the hospital administrator to manage doctors, patients and appointments.

The application is developed using **Kotlin and XML in Android Studio**, while **Supabase** (PostgreSQL cloud database) is used for authentication, data storage and management. The project implements important Mobile Application Development concepts such as user authentication, user interface design, database operations, screen navigation, role-based access, appointment management and prescription management.

The main objective of Medicare is to make the consultation process more convenient for patients and easier to manage for doctors and hospital administrators. The project also provides practical experience in developing an Android application and integrating different components into a functional mobile-based solution.

Overall, Medicare demonstrates the application of mobile technology to a practical real-world problem and provides a foundation that can be further enhanced with additional features in the future.

INDEX

Sr no	Content	Page
1.	Introduction	1
2.	Objectives	3
3.	Scope Of Project	4
4.	System Design	6
5.	Code	10
6.	Output	20
7.	Advantages	23
8.	Future Scope	24
9.	Conclusion	26
10.	Reference	27

INTRODUCTION

Getting medical consultation on time has become a common challenge in hospitals and clinics due to the increasing number of patients. Patients often have to visit the hospital personally, stand in long queues and wait for a long time only to book an appointment with a doctor. Traditional hospital management methods depend on manual registers and phone calls, which makes it difficult for patients to know which doctor is available and when.

In many situations, patients may not know which doctor is suitable for their problem or when the doctor is available. Doctors also need an efficient method to manage their appointments and maintain prescriptions, while hospital administrators need a simple way to manage doctors, patients and appointments. Therefore, a centralized mobile application can connect patients, doctors and the hospital administration digitally.

Medicare – Hospital Management and Doctor Appointment Application is an Android-based mobile application developed to provide a simple and convenient platform for searching doctors, booking appointments, issuing prescriptions and managing hospital records. The main purpose of the application is to reduce waiting time and provide a digital connection between patients, doctors and the hospital administration.

The application allows users to register and log in using their mobile number and password, search doctors by name or specialty, view doctor details and book an appointment by selecting a date and time slot. Doctors can accept, reject or complete appointment requests and add digital prescriptions, while patients can view their appointments and prescriptions. A separate admin module allows the hospital administrator to manage doctors, patients and appointments.

The application is developed using **Kotlin** for application logic, **XML** for user interface design and **Supabase** (PostgreSQL cloud database) for authentication, data storage and image storage. Android Studio and the Android SDK are used as the development environment.

Background of the Project

Hospitals and clinics handle a large number of patients every day. Managing doctor schedules, patient records, appointments and prescriptions manually requires a lot of time and effort and can easily lead to mistakes such as double bookings or lost records.

Traditional hospital management may involve paper registers for appointments and handwritten prescriptions. Patients may lose their prescriptions, and doctors may find it difficult to keep track of pending and completed appointments. A digital hospital management system can organize these activities and provide all the required information through a single mobile application.

Medicare provides a centralized solution where patients, doctors and administrators use the same Android application with separate role-based dashboards. All the data is stored securely in a cloud database, so the information is available instantly to the right user. The system is designed with a simple and user-friendly interface so that all users can easily understand and use it.

Problem Statement

Patients may face difficulty in finding suitable doctors and booking appointments, and hospitals may face difficulty in managing doctors, patients and appointments efficiently.

The existing manual hospital process may have the following difficulties:

1. Patients may not know which doctor is available before visiting the hospital.
2. Booking an appointment may require visiting the hospital or standing in long queues.
3. Appointment records may be maintained manually in registers.
4. Patients may not have a convenient method to view their appointment status.
5. Handwritten prescriptions may be lost or difficult to read.
6. Doctors may find it difficult to manage pending and completed appointments.
7. Hospital administrators may find it difficult to manage doctor and patient records.
8. Hospital information may not be organized in a centralized system.

Therefore, a mobile application is proposed to provide centralized hospital management and doctor appointment booking facilities.

OBJECTIVES

Main Objective

The main objective of Medicare is to develop an Android-based hospital management application that allows patients to find doctors and book appointments conveniently while providing doctors and hospital administrators with a digital system for managing appointments, prescriptions and hospital records.

Specific Objectives

The objectives of the project are:

1. To develop a user-friendly Android application for hospital management.
2. To allow users to register and log in securely using mobile number and password.
3. To provide separate dashboards for patients, doctors and administrators.
4. To allow patients to search doctors by name and specialty.
5. To display doctor details such as qualification, experience and consultation fee.
6. To allow patients to book appointments by selecting date and time slot.
7. To allow patients to view and cancel their appointments.
8. To allow doctors to accept, reject and complete appointment requests.
9. To allow doctors to issue digital prescriptions to patients.
10. To allow patients to view their prescriptions digitally.
11. To allow the admin to manage doctors, patients and appointments.
12. To use Supabase for storing and managing application data in the cloud.
13. To provide a simple and interactive mobile interface using Kotlin and XML.
14. To allow users to manage their profile and profile photo.

SCOPE OF THE PROJECT

Scope

The scope of Medicare is to provide a mobile-based solution for managing doctors, patients, appointments and prescriptions. The application focuses on making the consultation process easier for patients, doctors and hospital administrators.

The major areas covered by the application are:

3.1 Patient Side

The application allows patients to:

- Register an account and log in.
- Search doctors by name or specialty.
- View doctor details.
- Book an appointment by selecting date and time slot.
- View and cancel appointments.
- View prescriptions issued by the doctor.
- Manage their profile and profile photo.

3.2 Doctor Side

The application provides doctors with a separate dashboard.

Doctors can:

- Log in using doctor credentials.
- View pending, confirmed and completed appointments.
- Accept or reject appointment requests.
- Mark appointments as completed.
- Add prescriptions for patients.
- Manage their professional profile.

3.3 Admin Side

The hospital administrator can:

- View hospital statistics on the dashboard.
- Add and delete doctors.
- Manage patients.
- Manage appointments and change their status.

3.4 Database Scope

Supabase (PostgreSQL) is used for cloud data management. The database can maintain information related to:

- User profiles
- Doctors
- Patients
- Appointments
- Prescriptions

3.5 Limitations

The current application is mainly designed as an academic Mobile Application Development project.

Some advanced real-world facilities may require additional development, such as:

- Online payment gateway integration.
- Video consultation with doctors.
- Push notifications and reminders.
- Medical report and lab test uploads.
- Pharmacy and medicine ordering.
- Large-scale multi-hospital deployment.

These features can be considered for future enhancement.

SYSTEM DESIGN

4.1 System Overview

Medicare is an Android-based hospital management application developed using Kotlin and XML. The application provides three major types of users:

1. **Patient**
2. **Doctor**
3. **Admin**

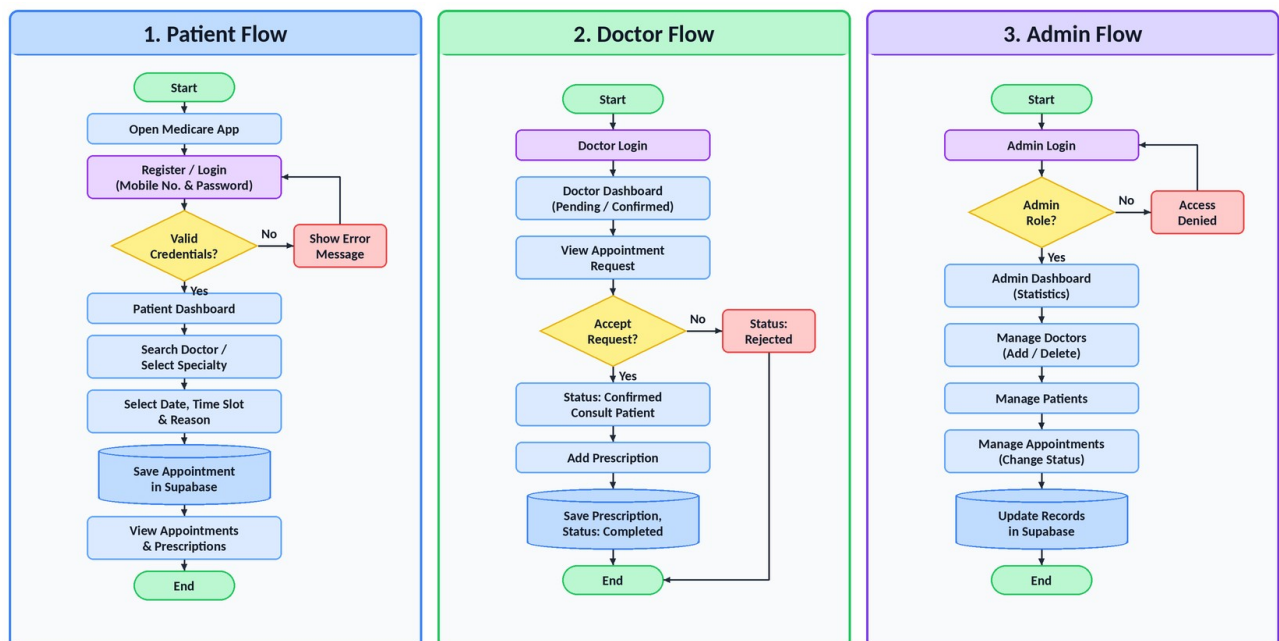
The patient interacts with the application to find doctors, book appointments and view prescriptions.

The doctor uses the doctor dashboard to manage appointments and issue prescriptions.

The admin uses the admin dashboard to manage doctors, patients and appointments.

The application uses Supabase to store important application information in the cloud.

Basic System Flow :



4.2 System Architecture

The system can be divided into the following layers:

i)Presentation Layer

It is developed using:

- XML
- Activities
- View Binding
- RecyclerView and CardView
- BottomNavigationView
- Material Components (Chips, TextInputLayout, Buttons)
- Dialogs and Animations

ii)Application Logic Layer

This layer contains the Kotlin code responsible for:

- User login and registration.
- Role-based navigation.
- Doctor search and filtering.
- Appointment booking.
- Appointment status management.
- Prescription management.
- Admin operations.
- Data validation.

iii) Database Layer

Supabase (PostgreSQL) is used for storing application data, Supabase Auth is used for authentication and Supabase Storage is used for profile images.

The database manages information related to:

- Profiles
- Doctors
- Patients
- Appointments
- Prescriptions

iv) Repository Layer

Repository classes (Auth, Doctor, Patient, Appointment, Prescription and Admin) use Kotlin Coroutines to communicate with Supabase in the background.

4.3 Main Components

The major components of Medicare are:

1.Authentication Module

Handles registration, login and session management for all users.

2.Patient Module

Allows patients to search doctors, book appointments and view prescriptions.

3.Doctor Module

Allows doctors to manage appointment requests and their profile.

4.Appointment Module

Creates appointments and maintains their status.

5.Prescription Module

Allows doctors to issue prescriptions and patients to view them.

6.Admin Module

Provides the administrator with management functions and statistics.

7.Database Module

Stores and retrieves application information using Supabase.

4.4 Database Design

Supabase (PostgreSQL) is used as the cloud database management system.

The major logical entities include:

i)Profiles

Stores user account information.

Information includes:

- Profile ID
- Name
- Email
- Phone
- Role (patient / doctor / admin)

ii)Doctors

Stores doctor information.

Information includes:

- Doctor ID
- Profile ID
- Specialization
- Qualification and experience
- Consultation fee
- Available days and timings

iii)Patients

Stores patient information.

Information includes:

- Patient ID
- Profile ID
- Date of birth and gender
- Blood group
- Address and emergency contact

iv)Appointments

Stores appointment information.

Information includes:

- Appointment ID
- Patient ID
- Doctor ID
- Appointment date and time
- Reason
- Status (pending / confirmed / rejected / completed / cancelled)

v)Prescriptions

Stores prescription information.

Information includes:

- Prescription ID
- Appointment ID
- Patient ID and Doctor ID
- Medicine and dosage
- Frequency and instructions

CODE

- **SplashActivity.kt**

```
package com.example.medicare.ui.splash

class SplashActivity : AppCompatActivity() {

    private lateinit var binding: ActivitySplashBinding
    private lateinit var sessionManager: SessionManager

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivitySplashBinding.inflate(layoutInflater)
        setContentView(binding.root)

        sessionManager = SessionManager(this)
        startAnimations()
        navigateNext()
    }

    private fun startAnimations() {
        binding.ivSplashLogo.apply {
            alpha = 0f
            scaleX = 0.75f
            scaleY = 0.75f
            animate().alpha(1f).scaleX(1.0f).scaleY(1.0f)
                .setDuration(700)
                .setInterpolator(AccelerateDecelerateInterpolator())
                .start()
        }
    }

    private fun navigateNext() {
        Handler(Looper.getMainLooper()).postDelayed({
            if (sessionManager.isLoggedIn()) {
                val role = sessionManager.getUserRole()
                val intent = when (role.lowercase()) {
                    "doctor" -> Intent(this, DoctorDashboardActivity::class.java)
                    "admin" -> Intent(this, AdminDashboardActivity::class.java)
                    else -> Intent(this, PatientDashboardActivity::class.java)
                }
                startActivity(intent)
            } else {
                startActivity(Intent(this, WelcomeActivity::class.java))
            }
            finish()
        }, 2000)
    }
}
```

- **LoginActivity.kt**

```
package com.example.medicare.ui.auth
```

```
class LoginActivity : AppCompatActivity() {

    private val authRepository = AuthRepository()
    private lateinit var sessionManager: SessionManager

    private fun setupListeners() {
        binding.btnLoginSubmit.setOnClickListener {
            val phone = binding.etLoginPhone.text?.toString()?.trim().orEmpty()
            val password = binding.etLoginPassword.text.toString().trim()
            if (phone.isEmpty()) {
                binding.tilLoginPhone.error = "Please enter your mobile number"
                return@setOnClickListener
            }
            if (password.isEmpty()) {
                binding.tilLoginPassword.error = "Please enter your password"
                return@setOnClickListener
            }
            performLogin(phone, password)
        }
    }

    private fun performLogin(phone: String, pass: String) {
        lifecycleScope.launch {
            val result = authRepository.signIn(phone, pass)
            result.fold(
                onSuccess = { profile ->
                    sessionManager.saveUserSession(profile)
                    navigateToDashboard(profile.role)
                },
                onFailure = { error ->
                    DialogUtils.showError(this@LoginActivity,
                        "Login Failed", error.localizedMessage)
                }
            )
        }
    }

    private fun navigateToDashboard(role: String) {
        val intent = when (role.lowercase()) {
            "doctor" -> Intent(this, DoctorDashboardActivity::class.java)
            "admin" -> Intent(this, AdminDashboardActivity::class.java)
            else -> Intent(this, PatientDashboardActivity::class.java)
        }
        startActivity(intent)
        finish()
    }
}
```

- **AuthRepository.kt**

```
package com.example.medicare.data.repository

/** Registration logic (called from RegisterActivity) */
suspend fun signUp(
    name: String,
    phone: String,
    userPassword: String,
    role: String,
    extraData: Map<String, String> = emptyMap()
): Result<Profile> = withContext(Dispatchers.IO) {
    try {
        val cleanPhone = normalizePhone(phone)
        val syntheticEmail = phoneToAuthEmail(cleanPhone)

        // Register user with Supabase Auth
        auth.signUpWithEmail {
            this.email = syntheticEmail
            this.password = userPassword
            data = buildJsonObject {
                put("name", name)
                put("phone", cleanPhone)
                put("role", role)
            }
        }

        val uid = auth.currentUserOrNull()?.id
        ?: return@withContext Result.failure(
            Exception("Registration completed, please sign in. "))

        // Save profile in the profiles table
        val newProfile = Profile(id = uid, name = name,
            email = syntheticEmail, phone = cleanPhone, role = role)
        db.from("profiles").insert(newProfile)

        // Create doctor or patient record
        if (role.equals("doctor", ignoreCase = true)) {
            ensureDoctorRecordExists(uid, name, extraData)
        } else if (role.equals("patient", ignoreCase = true)) {
            ensurePatientRecordExists(uid, extraData)
        }
        Result.success(newProfile)
    } catch (e: Exception) {
        Result.failure(e)
    }
}
```

- **PatientDashboardActivity.kt**

```
package com.example.medicare.ui.patient
```

```
class PatientDashboardActivity : AppCompatActivity() {

    private lateinit var binding: ActivityPatientDashboardBinding
    private lateinit var sessionManager: SessionManager

    private fun setupListeners() {
        // Search doctors
        binding.cardSearchBar.setOnClickListener {
            startActivity(Intent(this, DoctorListActivity::class.java))
        }

        // See all doctors
        binding.tvSeeAllDoctors.setOnClickListener {
            startActivity(Intent(this, DoctorListActivity::class.java))
        }

        // Specialities
        binding.catCardiology.setOnClickListener {
            openSpecialty("Cardiology")
        }
        binding.catNeurology.setOnClickListener {
            openSpecialty("Neurology")
        }
        binding.catPediatrics.setOnClickListener {
            openSpecialty("Pediatrics")
        }
        binding.catOrthopedics.setOnClickListener {
            openSpecialty("Orthopedics")
        }

        // Profile
        binding.btnPatientProfileAvatar.setOnClickListener {
            startActivity(Intent(this, PatientProfileActivity::class.java))
        }
    }

    private fun openSpecialty(specialty: String) {
        val intent = Intent(this, DoctorListActivity::class.java)
        intent.putExtra("SPECIALTY", specialty)
        startActivity(intent)
    }

    override fun onResume() {
        super.onResume()
        loadUpcomingAppointment()
        loadTopDoctors()
    }
}
```

- **BookAppointmentActivity.kt**

```
package com.example.medicare.ui.patient
```

```
class BookAppointmentActivity : AppCompatActivity() {

    private val appointmentRepository = AppointmentRepository()
    private var selectedDate: String = ""
    private var selectedTime: String = "09:30 AM"

    private fun setupListeners() {
        binding.etBookDate.setOnClickListener { showDatePicker() }

        binding.chipGroupTimeSlots.setOnCheckedChangeListener { _, ids ->
            selectedTime = when {
                ids.contains(R.id.chipSlot1) -> "09:30 AM"
                ids.contains(R.id.chipSlot2) -> "11:00 AM"
                ids.contains(R.id.chipSlot3) -> "02:30 PM"
                else -> "10:00 AM"
            }
        }

        binding.btnConfirmBooking.setOnClickListener {
            val reason = binding.etBookReason.text?.toString()?.trim().orEmpty()
            if (reason.isEmpty()) {
                binding.tilBookReason.error = "Please enter the reason"
                return@setOnClickListener
            }
            val patientId = sessionManager.getPatientId() ?: ""
            performBooking(patientId, doctorId, selectedDate, selectedTime, reason)
        }
    }

    private fun performBooking(patientId: String, docId: String,
                               date: String, time: String, reason: String) {
        lifecycleScope.launch {
            val result = appointmentRepository.createAppointment(
                patientId, docId, date, time, reason
            )
            result.fold(
                onSuccess = {
                    DialogUtils.showSuccess(this@BookAppointmentActivity,
                        "Appointment Booked!", "Booked on $date at $time")
                },
                onFailure = { error ->
                    DialogUtils.showError(this@BookAppointmentActivity,
                        "Booking Error", error.localizedMessage)
                }
            )
        }
    }
}
```

- **DoctorDashboardActivity.kt**

```
package com.example.medicare.ui.doctor
```

```
class DoctorDashboardActivity : AppCompatActivity() {

    private val appointmentRepository = AppointmentRepository()
    private lateinit var appointmentAdapter: DoctorAppointmentAdapter

    private fun setupUI() {
        appointmentAdapter = DoctorAppointmentAdapter(
            appointments = emptyList(),
            onAcceptClick = { apt -> updateStatus(apt.id, "confirmed") },
            onRejectClick = { apt -> updateStatus(apt.id, "rejected") },
            onCompleteClick = { apt -> updateStatus(apt.id, "completed") },
            onAddPrescriptionClick = { apt ->
                val intent = Intent(this, AddPrescriptionActivity::class.java)
                intent.putExtra("APPOINTMENT_ID", apt.id)
                intent.putExtra("PATIENT_ID", apt.patientId)
                intent.putExtra("DOCTOR_ID", apt.doctorId)
                startActivity(intent)
            }
        )
        binding.rvDoctorDashboardApts.layoutManager =
            LinearLayoutManager(this)
        binding.rvDoctorDashboardApts.adapter = appointmentAdapter
    }

    private fun loadDoctorAppointments() {
        val doctorId = sessionManager.getUserId() ?: return
        lifecycleScope.launch {
            val result = appointmentRepository.getDoctorAppointments(doctorId)
            result.onSuccess { list ->
                binding.tvPendingCount.text =
                    list.count { it.status == "pending" }.toString()
                binding.tvConfirmedCount.text =
                    list.count { it.status == "confirmed" }.toString()
                binding.tvCompletedCount.text =
                    list.count { it.status == "completed" }.toString()
                appointmentAdapter.updateData(list)
            }
        }
    }

    private fun updateStatus(appointmentId: String, newStatus: String) {
        lifecycleScope.launch {
            appointmentRepository.updateAppointmentStatus(
                appointmentId, newStatus
            )
            loadDoctorAppointments()
        }
    }
}
```

- **AddPrescriptionActivity.kt**

```
package com.example.medicare.ui.doctor
```

```
class AddPrescriptionActivity : AppCompatActivity() {

    private val prescriptionRepository = PrescriptionRepository()
    private var appointmentId = ""; private var patientId = ""
    private var doctorId = ""

    private fun setupListeners() {
        binding.btnSavePrescription.setOnClickListener {
            val medicine = binding.etRxMedicine.text.toString().trim()
            val dosage = binding.etRxDosage.text.toString().trim()
            val frequency = binding.etRxFrequency.text.toString().trim()
            val instructions = binding.etRxInstructions.text.toString().trim()

            if (medicine.isEmpty()) {
                binding.tilRxMedicine.error = "Medicine name is required"
                return@setOnClickListener
            }
            if (dosage.isEmpty()) {
                binding.tilRxDosage.error = "Dosage is required (e.g. 500mg)"
                return@setOnClickListener
            }
            savePrescription(medicine, dosage, frequency, instructions)
        }
    }

    private fun savePrescription(med: String, dos: String,
                                freq: String, inst: String) {
        lifecycleScope.launch {
            val result = prescriptionRepository.createPrescription(
                appointmentId = appointmentId, patientId = patientId,
                doctorId = doctorId, medicine = med, dosage = dos,
                frequency = freq, instructions = inst)
            result.fold(
                onSuccess = {
                    DialogUtils.showSuccess(this@AddPrescriptionActivity,
                        "Prescription Issued", "Saved successfully") { finish() }
                },
                onFailure = { error ->
                    DialogUtils.showError(this@AddPrescriptionActivity, "Error",
                        error.localizedMessage)
                }
            )
        }
    }
}
```

- **AdminDashboardActivity.kt**

```
package com.example.medicare.ui.admin
```

```
class AdminDashboardActivity : AppCompatActivity() {

    private val adminRepository = AdminRepository()
    private val authRepository = AuthRepository()
    private lateinit var sessionManager: SessionManager

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityAdminDashboardBinding.inflate(layoutInflater)
        setContentView(binding.root)
        sessionManager = SessionManager(this)
        setupListeners()
        loadStats()
    }

    private fun setupListeners() {
        binding.cardManageDoctors.setOnClickListener {
            startActivity(Intent(this, AdminManageDoctorsActivity::class.java))
        }
        binding.cardManagePatients.setOnClickListener {
            startActivity(Intent(this, AdminManagePatientsActivity::class.java))
        }
        binding.cardManageAppointments.setOnClickListener {
            startActivity(Intent(this, AdminManageAppointmentsActivity::class.java))
        }
        binding.btnAdminLogout.setOnClickListener {
            lifecycleScope.launch {
                authRepository.signOut()
                sessionManager.clearSession()
                startActivity(Intent(this@AdminDashboardActivity, LoginActivity::class.java))
                finish()
            }
        }
    }

    private fun loadStats() {
        lifecycleScope.launch {
            adminRepository.getDashboardStats().onSuccess { stats ->
                binding.tvAdminTotalDoctors.text = stats.totalDoctors.toString()
                binding.tvAdminTotalPatients.text = stats.totalPatients.toString()
                binding.tvAdminTotalAppointments.text = stats.totalAppointments.toString()
                binding.tvAdminPendingApts.text = stats.pendingAppointments.toString()
            }
        }
    }
}
```

- **SupabaseManager.kt**

```
package com.example.medicare.data
```

```
import io.github.jan.supabase.SupabaseClient
import io.github.jan.supabase.createSupabaseClient
import io.github.jan.supabase.gotrue.Auth
import io.github.jan.supabase.postgrest.Postgrest
import io.github.jan.supabase.storage.Storage
import io.github.jan.supabase.realtime.Realtime
```

```
object SupabaseManager {
```

```
    lateinit var client: SupabaseClient
    private set
```

```
    val auth: Auth
        get() = client.auth
```

```
    val db: Postgrest
        get() = client.postgrest
```

```
    val storage: Storage
        get() = client.storage
```

```
    fun initialize(context: Context) {
        // Read credentials from BuildConfig
        val targetUrl = BuildConfig.SUPABASE_URL.trim()
        val targetKey = BuildConfig.SUPABASE_ANON_KEY.trim()
```

```
        require(targetUrl.isNotBlank()) { "SUPABASE_URL is empty" }
        require(targetKey.isNotBlank()) { "SUPABASE_ANON_KEY is empty" }
```

```
        client = createSupabaseClient(
            supabaseUrl = targetUrl,
            supabaseKey = targetKey
        ) {
            install(Auth)
            install(Postgrest)
            install(Storage)
            install(Realtime)
        }
    }
```

```
}
```

- **AndroidManifest.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission
        android:name="android.permission.ACCESS_NETWORK_STATE" />

    <application
        android:name=".MedicareApp"
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.Medicare">

        <activity
            android:name=".ui.splash.SplashActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
                    android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".ui.auth.WelcomeActivity" />
        <activity android:name=".ui.auth.LoginActivity" />
        <activity android:name=".ui.auth.RegisterActivity" />

        <activity android:name=".ui.patient.PatientDashboardActivity" />
        <activity android:name=".ui.patient.DoctorListActivity" />
        <activity android:name=".ui.patient.BookAppointmentActivity" />
        <activity android:name=".ui.patient.MyAppointmentsActivity" />
        <activity android:name=".ui.patient.PrescriptionActivity" />

        <activity android:name=".ui.doctor.DoctorDashboardActivity" />
        <activity android:name=".ui.doctor.AddPrescriptionActivity" />
        <activity android:name=".ui.doctor.DoctorProfileActivity" />

        <activity android:name=".ui.admin.AdminDashboardActivity" />
        <activity android:name=".ui.admin.AdminManageDoctorsActivity" />
        <activity android:name=".ui.admin.AdminManagePatientsActivity" />
        <activity
            android:name=".ui.admin.AdminManageAppointmentsActivity" />
    </application>
</manifest>
```

OUTPUT:

[Paste screenshot here]

Login Screen

Login Screen

[Paste screenshot here]

Patient Dashboard

Patient Dashboard

[Paste screenshot here]

Doctor List Screen

[Paste screenshot here]

Book Appointment Screen

Doctor List Screen

Book Appointment Screen

[Paste screenshot here]

Doctor Dashboard

[Paste screenshot here]

Admin Dashboard

Doctor Dashboard

Admin Dashboard

ADVANTAGES

1. **Easy Doctor Search**
Patients can search doctors by name and specialty through the application.
2. **Convenient Appointment Booking**
Patients can book appointments by selecting a date and time slot.
3. **Reduced Waiting Time**
Online booking reduces the time spent in queues at the hospital.
4. **Role-Based Access**
Patients, doctors and admins get separate dashboards and functions.
5. **Appointment Management**
Doctors can accept, reject and complete appointment requests.
6. **Digital Prescriptions**
Prescriptions are stored digitally and cannot be lost.
7. **Admin Control**
The admin can manage doctors, patients and appointments from one place.
8. **Centralized Information**
Hospital-related information can be accessed through a single application.
9. **User-Friendly Interface**
The application provides a simple mobile interface for all users.
10. **Cloud Database Support**
Supabase stores data securely in the cloud and keeps it up to date.
11. **Secure Login**
Supabase Authentication and row level security protect user data.
12. **Android-Based Solution**
Users can access the system through an Android mobile application.

FUTURE SCOPE :

1. Online Payment
An online payment gateway can be integrated so that patients can pay the consultation fee directly through the application.
2. Video Consultation
Patients can consult doctors through video calls from home.
3. Medical Reports
Patients can upload and view lab reports, X-rays and other medical documents.
4. Online Pharmacy
Medicines from the prescription can be ordered directly from a pharmacy.
5. Multi-Hospital Support
The application can be extended to manage multiple hospitals and clinics.
6. Multi-Language Support
The application can be made available in regional languages such as Marathi and Hindi.
7. Doctor Ratings
Patients can rate doctors and write reviews after consultation.
8. Notifications
Push notifications can be added for:
 - Appointment confirmation.
 - Appointment reminders.
 - Appointment cancellation.
 - New prescription issued.
9. Admin Analytics
Administrators can be provided with analytical information such as:
 - Number of appointments.
 - Most consulted doctors.
 - Department-wise patient count.
 - Daily and monthly appointment reports.

ACHIEVED COURSE OUTCOMES:

- CO1: Describe features of Android operating system.
- CO2: Setup and configure the Android development environment.
- CO3: Design user-friendly graphical interfaces
- CO4 : Build Android applications using standard UI components.
- CO5: Develop Android applications with integrated database functionality
- CO6: Deploy Android application.

CONCLUSION

The Medicare application successfully provides a simple and organized solution for hospital management and doctor appointment booking through a mobile application. The main purpose of the project is to reduce the difficulties faced by patients while finding doctors and booking appointments and to provide a convenient method for managing hospital records.

The application is developed using Android Studio with Kotlin and XML, along with Supabase for authentication and for storing and managing application data in the cloud. The use of these technologies helped in implementing important mobile application development concepts such as user authentication, database connectivity, activity and screen navigation, user interface design, appointment management and prescription management.

Medicare provides separate functionality for patients, doctors and administrators. Patients can search doctors and book appointments, doctors can manage appointments and issue prescriptions, and the admin can manage doctors, patients and appointments. This separation helps in making the application more organized and suitable for practical hospital management.

The project also helped in understanding how different components of an Android application work together. The implementation of Supabase provided practical knowledge of cloud data storage and database operations, while Kotlin was used to handle application logic and user interactions. XML was used to design the application interfaces and provide a structured and user-friendly experience.

Overall, Medicare demonstrates how mobile application development concepts can be applied to solve a real-world problem. The project provides a foundation that can be further improved with additional features and technologies in the future.

Thus, the Medicare project successfully meets its primary objective of developing a mobile-based hospital management and appointment booking application while providing practical knowledge and experience in Android application development, Kotlin programming, XML-based UI design, cloud database management, and application integration.

REFERENCES

1. Android Developers, **Android Developer Documentation**, Google.
2. Android Developers, **Activities and Activity Lifecycle**, Google.
3. Android Developers, **User Interface and Layouts**, Google.
4. Kotlin Documentation, **Kotlin Programming Language**, JetBrains.
5. Kotlin Documentation, **“Coroutines Guide,”** JetBrains.
6. Supabase, **“Supabase Documentation,”** Supabase Inc.
7. Supabase, **Supabase Kotlin Client Library (supabase-kt)**.
8. PostgreSQL, **PostgreSQL Documentation**.
9. Material Design, **Material Components for Android**, Google.
10. Android Studio, **Integrated Development Environment for Android Application Development**, Google.